



FORM BUILDER & RESPONSE MANAGEMENT SYSTEM

DETAILED PRODUCTION ARCHITECTURE DOCUMENT (ENTERPRISE LEVEL)

Naiyo24 Private Limited – Internal Engineering Execution Document

1. PROJECT OVERVIEW

This project is to build a **fully scalable, production-grade Form Builder & Response Management System** similar to Google Forms but optimized for **performance, flexibility, and extensibility**.

The system will allow:

- Dynamic form creation (drag & drop)
- Public/private form sharing
- Secure response collection
- Export (PDF, Excel)
- Dashboard with analytics

This is not a basic CRUD project.

This is a **dynamic schema-driven system** where forms are created at runtime.

2. CORE OBJECTIVES

- Build a **schema-driven form engine**
 - Support **high-volume submissions**
 - Ensure **secure data handling**
 - Enable **fast export generation**
 - Design for **future extensibility (AI, analytics, automation)**
-

3. HIGH-LEVEL SYSTEM ARCHITECTURE

Components:

- Flutter Frontend (Builder + Dashboard + Public Form UI)
 - FastAPI Backend (Core logic)
 - PostgreSQL (Structured data)
 - Redis (Caching)
 - Object Storage (File uploads)
 - Worker Queue (Export & async jobs)
-

4. COMPLETE SYSTEM FLOW

4.1 Form Creation Flow

1. User opens builder UI
 2. Adds fields dynamically
 3. Form converted into JSON schema
 4. Schema validated
 5. Stored in DB
 6. Public URL generated
-

4.2 Form Rendering Flow

1. User opens public link
 2. Backend fetches form schema
 3. Flutter dynamically renders UI
 4. Validations applied
-

4.3 Response Submission Flow

1. User fills form
2. Data validated (frontend + backend)
3. Stored as JSON response
4. Increment response count (Redis)

4.4 Export Flow

1. User requests export
 2. Background worker triggered
 3. Data processed
 4. PDF/Excel generated
 5. File returned
-

5. BACKEND ARCHITECTURE (FASTAPI – FULL BREAKDOWN)

COMPLETE FILE STRUCTURE

```
backend/
├── app/
│   ├── main.py
│   ├── core/
│   │   ├── config.py           # Env configs
│   │   ├── database.py        # DB connection
│   │   ├── security.py        # JWT/Auth
│   │   └── dependencies.py
│   ├── api/
│   │   ├── routes/
│   │   │   ├── form.py        # Form CRUD
│   │   │   ├── response.py    # Submit responses
│   │   │   ├── export.py      # Export APIs
│   │   │   └── auth.py
│   │   └── router.py
│   ├── models/
│   │   ├── user.py
│   │   ├── form.py
│   │   ├── form_field.py
│   │   └── response.py
│   ├── schemas/
│   │   ├── form_schema.py
│   │   └── response_schema.py
│   └── services/
│       └── form_service.py
```

```

|   |   | response_service.py
|   |   | export_service.py
|   |   | validation_service.py
|
|   |   | utils/
|   |   | | schema_engine.py           # Dynamic schema logic
|   |   | | validators.py
|   |   | | logger.py
|   |
|   |   | workers/
|   |   | | celery_worker.py
|   |   | | export_tasks.py
|
| migrations/
| tests/
| Dockerfile
| docker-compose.yml
| requirements.txt
| .env

```

6. FRONTEND ARCHITECTURE (FLUTTER – FULL BREAKDOWN)

```

flutter_app/
|
|   | lib/
|   | | main.dart
|   | | app.dart
|   |
|   |   | core/
|   |   | | constants/
|   |   | | theme/
|   |   | | utils/
|   |   | | network/
|   |
|   |   | services/
|   |   | | api_service.dart
|   |   | | form_service.dart
|   |   | | response_service.dart
|   |
|   |   | providers/
|   |   | | form_provider.dart
|   |   | | response_provider.dart
|   |
|   |   | features/
|   |   | | builder/
|   |   | | | presentation/
|   |   | | | | screens/
|   |   | | | | | form_builder.dart
|   |   | | | | widgets/
|   |   | | | | field_selector.dart

```

```
|
|
|           |— drag_drop_area.dart
|
|   |— renderer/
|   |   |— presentation/
|   |   |   |— screens/
|   |   |   |   |— form_renderer.dart
|   |   |   |   |— widgets/
|   |   |   |   |   |— dynamic_field.dart
|   |   |
|   |   |— dashboard/
|   |   |   |— screens/
|   |   |   |   |— form_list.dart
|   |   |   |   |— response_dashboard.dart
|   |   |
|   |   |— export/
|   |   |   |— screens/
|   |   |   |   |— export_screen.dart
|
|   |— routes/
|   |— config/
|
|— assets/
|— pubspec.yaml
```

7. CORE ENGINE DESIGN (IMPORTANT)

Form Schema Example

```
{
  "title": "User Feedback",
  "fields": [
    {
      "type": "text",
      "label": "Name",
      "required": true
    },
    {
      "type": "email",
      "label": "Email"
    }
  ]
}
```

Why JSON Schema?

- Dynamic UI rendering
- No DB schema change needed
- Flexible form structure

8. DATABASE DESIGN (DETAILED)

forms

- id
- title
- description
- schema (JSON)
- created_by
- created_at

responses

- id
- form_id
- response_data (JSON)
- submitted_at

form_fields (optional normalized)

- id
- form_id
- field_type
- metadata

9. REDIS STRATEGY

- Cache form schema
- Cache response count
- Reduce DB load

10. EXPORT SYSTEM DESIGN

Excel

- Library: openpyxl
- Flatten JSON → rows

PDF

- Library: reportlab
 - Structured table format
-

11. SECURITY DESIGN

- Validate input fields
 - Prevent spam (rate limit)
 - CAPTCHA (future)
 - Sanitize user input
-

12. PERFORMANCE OPTIMIZATION

- Lazy load responses
 - Pagination
 - Background export
 - Redis caching
-

13. DEVELOPMENT ROADMAP

Phase 1

- Form builder
- Response collection

Phase 2

- Export system
- Dashboard

Phase 3

- Analytics
 - File upload field
-

14. TEAM EXECUTION PLAN

Frontend Team

- Builder UI
- Dynamic renderer
- Dashboard UI

Backend Team

- Schema engine
- Response APIs
- Export system

QA Team

- Submission flow
 - Export validation
 - Load testing
-

15. SUCCESS METRICS

- Forms created
 - Responses collected
 - Export usage
 - User retention
-

16. ADVANCED IMPLEMENTATION SUPPORT

Backend

- Complete FastAPI production code
- Dynamic schema engine

Frontend

- Drag-drop builder system

- Dynamic renderer

DevOps

- Domain + SSL
 - CI/CD pipeline
 - Docker production setup
-

17. FUTURE SCOPE

- Conditional logic
 - Email notifications
 - AI-based insights
 - Team collaboration
-

18. FINAL INSTRUCTION

This system must be built as a **core scalable product**.

Strict rules:

- Follow architecture
 - No shortcut coding
 - Maintain clean structure
 - Think for scale
-

END OF DOCUMENT